

Name: _____

CptS 355 Exam1 (Fall 2026)

Write your full name at the top of every page. When asked to explain or write prose, be very clear and concise, expressing your answer in a few sentences at most. Clear diagrams and mathematical notation are desirable wherever applicable, and partial solutions may earn partial credit.

- 1. (6 pts)** Draw an abstract syntax tree for the following regular expression:

$$a(b \mid \epsilon)^*c$$

Use a binary ($\mid$) node for disjunction, a binary ($\cdot$) node for sequencing (written by juxtaposition in the expression), and a unary ($*$) node. Treat ϵ and each letter as leaves.

- 2. (6 pts)** What is the Cartesian product $\{1, 2, 3\} \times \emptyset$?

3. (9 pts) In the posted code, `Char('a') == Char('a')` is false. Explain why. How would you implement structural equality for Char regexes?

4. (6 pts) Draw the abstract syntax tree (AST) for the following arithmetic expression. Put operators at internal nodes and numbers at leaves:

$$3 + 4 * 2$$

5. (6 pts) Let $e = \mathbf{a^*b}$. Compute and simplify $D_{\mathbf{a}}(e)$ and $D_{\mathbf{b}}(e)$. In each calculation, show the two branches of the sequence derivative rule and explain why both branches must be considered.

6. (10 pts) The posted matcher repeatedly replaces a regular expression e by $D_c(e)$ for the next character c , then returns `re.containsEmpty()`. State the invariant that relates the consumed prefix, the current residual expression, and the original language. Use it to explain why the final nullability check decides whole-string membership.

7. (4 pts) Rewrite the posted free function `matches(re, s)` as a method inherited by every regular-expression class. Fill in the body below and state whether calling the method mutates the original regular-expression object.

```
class Regex:
    def matches(self, s):
        # fill in
```

8. (5 pts) Draw the complete truth table for $\neg A \vee \neg B$. Then give an equivalent proposition without using disjunction.

9. (3 pts) True / False: Every mathematical function is also a relation.
10. (3 pts) True / False: An equivalence relation is transitive, reflexive and symmetric.
11. (3 pts) True / False: An equivalence relation is transitive, reflexive and anti-symmetric.
12. (3 pts) True / False: The intersection of two regular languages is always regular.
13. (3 pts) True/False: A graph can be encoded as a vertex set paired with an edges relation.
14. (3 pts) True or False: Procedural programming emphasizes using procedures as reusable black-box abstractions.
15. (3 pts) True/False: every partial function $f : A \rightarrow B$ is also a relation $f \subseteq A \times B$.
16. (3 pts) True or False: C++ is a good example of the structured, imperative, and object-oriented language paradigms.
17. (3 pts) True or False: Imperative programming is oriented around giving the computer commands that have an effect on the program's environment or the surrounding system environment.
18. (4 pts) Define $\mathbb{N}^4$ (i.e., $\mathbb{N} \times \mathbb{N} \times \mathbb{N} \times \mathbb{N}$), as precisely as you can, using set-builder notation.

19. (4 pts) Draw an abstract syntax tree for the following regular expression:

ab^*

20. (10 pts) Which strings are in the regular language $aa^*b|bb^*a$? (circle all that are)

aabba

aabb

ϵ

a

bbb

21. (3 pts) True / False: A graph that is irreflexive has no immediate self-loops at any vertex (no vertex is connected to itself).

22. (3 pts) True/False: A function f is surjective if and only if $f(x) = f(y) \implies x = y$.

23. (3 pts) True / False: A predicate is a proposition abstracted over an unknown variable.

24. (3 pts) True / False: A function f is monotonic if and only if $x \geq y \implies f(x) \geq f(y)$.

25. (6 pts) Draw the AST for the following arithmetic expression, then evaluate it:

$$2 \wedge 3 \wedge 2 - (5 - 1) * 3$$

This page intentionally left blank. Feel free to use this as scratch space.

Name: _____

CptS 355 Exam1 (Fall 2026)

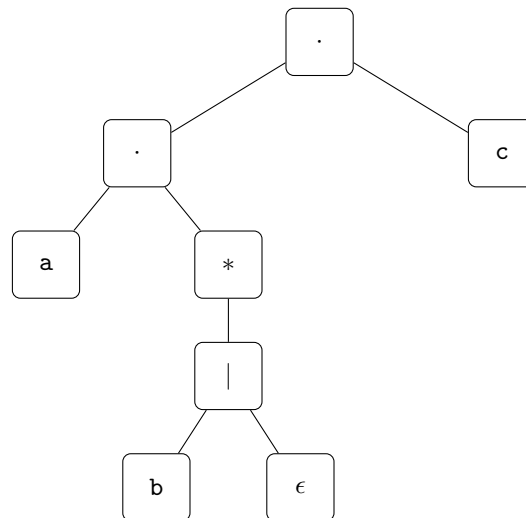
Write your full name at the top of every page. When asked to explain or write prose, be very clear and concise, expressing your answer in a few sentences at most. Clear diagrams and mathematical notation are desirable wherever applicable, and partial solutions may earn partial credit.

1. (6 pts) Draw an abstract syntax tree for the following regular expression:

$$a(b \mid \epsilon)^*c$$

Use a binary ($\mid$) node for disjunction, a binary ($\cdot$) node for sequencing (written by juxtaposition in the expression), and a unary ($*$) node. Treat ϵ and each letter as leaves.

Solution / grading guide. The parenthesized disjunction is the operand of star. Left-grouping the outer three-part sequence gives this tree:



2. (6 pts) What is the Cartesian product $\{1, 2, 3\} \times \emptyset$?

Solution / grading guide.

$$\{1, 2, 3\} \times \emptyset = \emptyset,$$

because there is no possible second value to form a pair with.

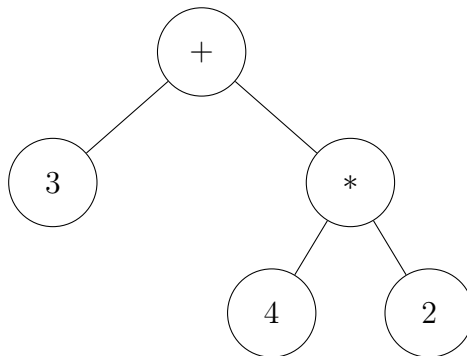
3. (9 pts) For a Python class `Char`, by default, `Char('a') == Char('a')` is false. Explain why. How would you implement structural equality for `Char` regexes?

Solution / grading guide. Without `__eq__`, Python objects compare by identity, so two separately constructed `Char` objects are distinct. Structural equality should compare the node type and corresponding fields recursively. A matching hash can be built by hashing all relevant properties together (the type and subexpressions). Equal objects must have equal hashes.

4. (6 pts) Draw the abstract syntax tree (AST) for the following arithmetic expression. Put operators at internal nodes and numbers at leaves:

$$3 + 4 * 2$$

Solution. Multiplication binds more tightly than addition.



5. (6 pts) Let $e = \mathbf{a^*b}$. Compute and simplify $D_{\mathbf{a}}(e)$ and $D_{\mathbf{b}}(e)$. In each calculation, show the two branches of the sequence derivative rule and explain why both branches must be considered.

Solution / grading guide. Since $\epsilon \in \mathcal{L}(\mathbf{a^*})$,

$$D_c(\mathbf{a^*b}) = D_c(\mathbf{a^*})\mathbf{b} \mid D_c(\mathbf{b}).$$

Therefore

$$\begin{aligned} D_{\mathbf{a}}(\mathbf{a^*b}) &= (\epsilon\mathbf{a^*})\mathbf{b} \mid \emptyset = \mathbf{a^*b}, \\ D_{\mathbf{b}}(\mathbf{a^*b}) &= (\emptyset\mathbf{a^*})\mathbf{b} \mid \epsilon = \epsilon. \end{aligned}$$

The character may be consumed either within the starred left operand or, because that operand may be empty, by the right operand.

6. (10 pts) The posted matcher repeatedly replaces a regular expression e by $D_c(e)$ for the next character c , then returns `re.containsEmpty()`. State the invariant that relates the consumed prefix, the current residual expression, and the original language. Use it to explain why the final nullability check decides whole-string membership.

Solution / grading guide. After consuming a prefix p , the current expression denotes

$$D_p(\mathcal{L}(e)) = \{ s \mid ps \in \mathcal{L}(e) \}.$$

In words, its language contains exactly the suffixes that could complete the already-consumed prefix to a string in the original language. When the whole input w has been consumed, no suffix remains. Therefore

$$w \in \mathcal{L}(e) \iff \epsilon \in D_w(\mathcal{L}(e)).$$

`containsEmpty()` implements precisely the right-hand test.

7. (4 pts) Rewrite the posted free function `matches(re, s)` as a method inherited by every regular-expression class. Fill in the body below and state whether calling the method mutates the original regular-expression object.

```
class Regex:
    def matches(self, s):
        # fill in
```

Solution / grading guide.

```
class Regex:
    def matches(self, s):
        re = self
        for c in s:
            re = re.D(c)
        return re.containsEmpty()
```

The local name `re` is rebound to newly constructed residual ASTs. The posted node methods do not change fields of `self`, so the original expression is not mutated. Equivalent code that iterates through the characters and tests nullability at the end receives full credit.

8. (5 pts) Draw the complete truth table for $\neg A \vee \neg B$. Then give an equivalent proposition without using disjunction.

Solution / grading guide.

A	B	$\neg A$	$\neg B$	$\neg A \vee \neg B$
F	F	T	T	T
F	T	T	F	T
T	F	F	T	T
T	T	F	F	F

By De Morgan's law,

$$\neg A \vee \neg B \iff \neg(A \wedge B).$$

9. (3 pts) **True** / False: Every mathematical function is also a relation.
10. (3 pts) **True** / False: An equivalence relation is transitive, reflexive and symmetric.
11. (3 pts) True / **False**: An equivalence relation is transitive, reflexive and anti-symmetric.
12. (3 pts) **True** / False: The intersection of two regular languages is always regular.
13. (3 pts) **True** / False: A graph can be encoded as a vertex set paired with an edges relation.
14. (3 pts) **True** or False: Procedural programming emphasizes using procedures as reusable black-box abstractions.
15. (3 pts) **True** / False: every partial function $f : A \rightarrow B$ is also a relation $f \subseteq A \times B$.
16. (3 pts) **True** or False: C++ is a good example of the structured, imperative, and object-oriented language paradigms.
17. (3 pts) **True** or False: Imperative programming is oriented around giving the computer commands that have an effect on the program's environment or the surrounding system environment.
18. (4 pts) Define $\mathbb{N}^4$ (i.e., $\mathbb{N} \times \mathbb{N} \times \mathbb{N} \times \mathbb{N}$), as precisely as you can, using set-builder notation.

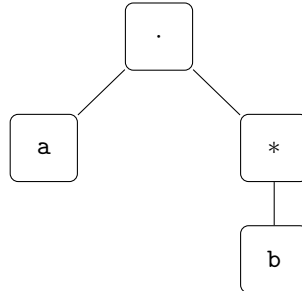
Solution / grading guide.

$$\mathbb{N}^4 = \{(a, b, c, d) \mid a, b, c, d \in \mathbb{N}\}.$$

19. (4 pts) Draw an abstract syntax tree for the following regular expression:

ab^*

Solution / grading guide. Star binds more tightly than juxtaposition, so the root is sequencing and the star applies only to b :



20. (10 pts) Which strings are in the regular language $aa^*b \mid bb^*a$? (circle all that are)

aabba

aabb

ϵ

a

bbb

Solution / grading guide. None of the listed strings match. The two branches require a^+b or b^+a : a nonempty run of one symbol followed by exactly one of the other.

21. (3 pts) **True** / False: A graph that is irreflexive has no immediate self-loops at any vertex (no vertex is connected to itself).

22. (3 pts) True/**False**: A function f is surjective if and only if $f(x) = f(y) \implies x = y$.

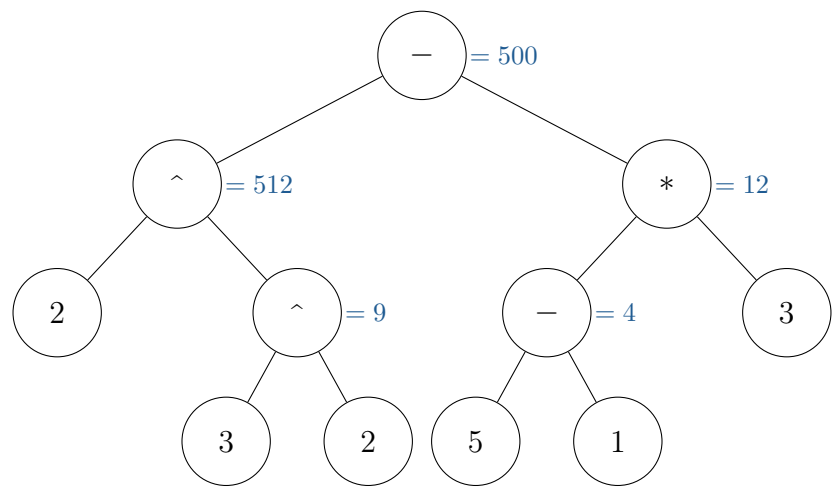
23. (3 pts) **True** / False: A predicate is a proposition abstracted over an unknown variable.

24. (3 pts) **True** / False: A function f is monotonic if and only if $x \geq y \implies f(x) \geq f(y)$.

25. (6 pts) Draw the AST for the following arithmetic expression, then evaluate it:

$$2 \wedge 3 \wedge 2 - (5 - 1) * 3$$

Solution.



This page intentionally left blank. Feel free to use this as scratch space.